

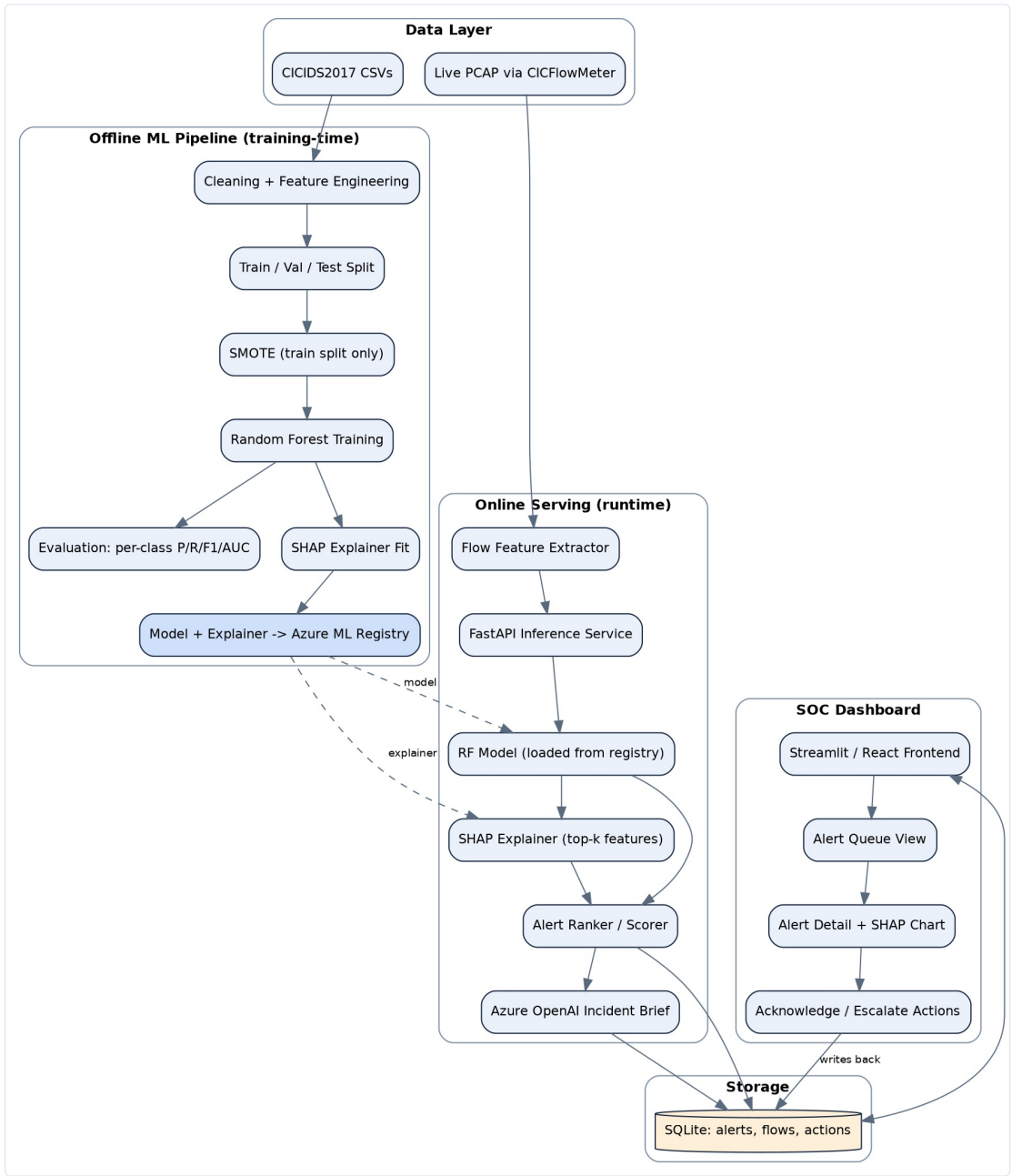
NetSentinel Architecture & Build Plan

NetSentinel — System Architecture & Build Plan

Project: NetSentinel — ML layer that flags what signature IDS misses, for SOC analysts

Scope of this doc: what to build, how the pieces fit together, and the exact task list to get from zero to a working demo.

1. Architecture Overview



NetSentinel Architecture

Two pipelines, not one. This is the single most important design decision:

- **Offline pipeline** (Section 3): runs once per model version, on your machine or in Azure ML. Produces a

trained model + fitted SHAP explainer, registered as versioned artifacts.

- **Online pipeline** (Section 5): runs per network flow, at inference time. Loads the registered model, scores traffic, explains, ranks, and writes to the alert queue.

Keeping these separate means retraining never touches the serving code, and the serving code never needs scikit-learn’s training dependencies (SMOTE, cross-validation, etc.) — smaller, faster, less to break during a demo.

2. Component Responsibilities

Component	Responsibility	Key libraries
Feature Extractor	Raw packets/CSV → flow-level feature vector	CICFlowMeter (live), pandas (CICIDS2017)
Preprocessing	Clean, encode, scale, handle class imbalance	pandas, scikit-learn, imbalanced-learn (SMOTE)
Model	Binary (normal/attack) + multiclass (attack type) classification	scikit-learn <code>RandomForestClassifier</code>
Explainability	Per-alert feature attribution	<code>shap</code> (<code>TreeExplainer</code>)
Model Registry	Versioned model + explainer artifacts	Azure Machine Learning
Inference API	Serve predictions, manage alert lifecycle	FastAPI, Uvicorn
Database	Persist flows, alerts, analyst actions	SQLite (dev)
Incident Brief Generator	Plain-English summary per alert	Azure OpenAI (<code>gpt-4o-mini</code> or similar)
Dashboard	SOC analyst-facing UI	Streamlit (fast path) or React (polish path)

3. Offline ML Pipeline — Detail

3.1 Data

- **Source:** CICIDS2017 (unb.ca) — 8 CSV files, one per day/attack scenario, ~78 flow-level features (from CICFlowMeter) + `Label` column.
- **Target design:**
 - `y_binary`: `BENIGN` → 0, everything else → 1
 - `y_multiclass`: original label, collapsed into a manageable set (e.g. DoS, DDoS, PortScan, Brute Force, Web Attack, Botnet, Infiltration, BENIGN) — CICIDS2017 has some ultra-rare classes (Heartbleed, Infiltration) that need to be merged into an “Other/Rare” bucket or excluded from the multiclass head, or the model will overfit to memorizing 1–2 examples.

3.2 Cleaning

- Drop columns that are pure identifiers/leakage: `Flow ID`, `Source IP`, `Destination IP`, `Timestamp` (keep IPs/timestamp in a separate lookup table for the dashboard, not as model features — IP addresses are not generalizable signal and will make the demo dataset look better than it is).
- Handle `Infinity` / `NaN` values (CICIDS2017 is known to have some malformed rows from the original CICFlowMeter export — drop or clip these, don’t silently zero-fill without checking how many rows are affected).
- Deduplicate identical flow rows.

3.3 Feature Engineering

- Standardize numeric ranges (RF doesn’t strictly need scaling, but keep a `StandardScaler` fitted and saved

anyway — you’ll want it if you ever add a linear/NN model for comparison, and Azure OpenAI’s brief-writing step benefits from human-readable, not scaled, feature values, so keep both raw and scaled versions available).

- Correlation check: drop near-duplicate features (CICIDS2017 has several highly correlated flow-duration variants) — keeps SHAP output less redundant and the model faster.

3.4 Split Strategy

- **Split by time or by capture day, not randomly.** Random row-level splitting on CICIDS2017 leaks information (flows from the same session end up in both train and test) and inflates your reported accuracy. Use day-based splits (e.g. train on Mon–Thu captures, test on Fri) and say so explicitly in your evaluation writeup — this is a strong technical-credibility point precisely because most student projects get this wrong.

3.5 Class Imbalance

- Apply SMOTE **only to the training split, after the split, never before** (SMOTE before splitting leaks synthetic near-duplicates into the test set and inflates recall).
- Apply SMOTE to the multiclass target with care — consider `SMOTE` only for classes with a reasonable number of real examples (roughly >20–30), and use class weights (`class_weight='balanced'`) instead of oversampling for the ultra-rare classes.

3.6 Model Training

- Two classifiers, trained separately:
 1. `RandomForestClassifier` — binary normal/attack
 2. `RandomForestClassifier` — multiclass attack-type (only invoked when binary head says “attack”)
- Hyperparameters to actually tune (not just defaults): `n_estimators` , `max_depth` , `min_samples_leaf` , `class_weight` . Use `RandomizedSearchCV` with a time-aware cross-validation split, not plain k-fold (same leakage reasoning as 3.4).
- Save: model files (`.joblib`), the exact feature column order (critical — inference must reconstruct features in identical order), and the label encoders.

3.7 Evaluation

- Report **per-class** precision, recall, F1, and macro-averaged AUC — not just overall accuracy (CICIDS2017 is >80% benign traffic; a model that always predicts “benign” scores ~80% accuracy and is useless).
- Confusion matrix, saved as an artifact, not just a number in a notebook.
- A short “known limitations” note in your own docs: trained/tested on one dataset, one point in time, lab-generated traffic — this honesty is a repeated ask in your hackathon brief and matters for the live demo Q&A too.

3.8 Explainability (SHAP)

- Fit `shap.TreeExplainer` on the trained RF (fast — no retraining needed, tree explainers are close to free for RF).
- At inference time, for each alert, extract the top 5 contributing features and their SHAP values — this is what feeds both the dashboard chart and the Azure OpenAI brief prompt.
- Precompute and cache global feature importance separately (for a “what does the model generally look at” chart on the dashboard, distinct from per-alert local explanations).

3.9 Registry (Azure Machine Learning)

- Create an Azure ML workspace (`az ml workspace create`).
- Register each trained artifact as a versioned model: `az ml model create` or the Python SDK v2 (`MLClient.models.create_or_update`), tagging with dataset version, split strategy, and eval metrics as metadata — so “which model is currently serving” is always traceable.
- The FastAPI service pulls the model by `name:version` (or `@latest`) at startup, not by a local hardcoded file path — this is what makes “Azure ML for model management” a true claim rather than a sticker.

4. Database Schema (SQLite, dev-stage)

```
-- raw/derived flow features, one row per observed flow
CREATE TABLE flows (
  flow_id      TEXT PRIMARY KEY,
  captured_at  DATETIME,
  src_ip       TEXT,
  dst_ip       TEXT,
  src_port     INTEGER,
  dst_port     INTEGER,
  protocol     TEXT,
  feature_json TEXT      -- serialized feature vector used by the model
);

-- one row per generated alert
CREATE TABLE alerts (
  alert_id     TEXT PRIMARY KEY,
  flow_id      TEXT REFERENCES flows(flow_id),
  model_version TEXT,
  is_attack    BOOLEAN,
  attack_type  TEXT,
  confidence   REAL,
  top_features TEXT,      -- serialized [{feature, shap_value}, ...]
  incident_brief TEXT,    -- Azure OpenAI generated text
  status       TEXT DEFAULT 'new', -- new | acknowledged | escalated | dismissed
  created_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at   DATETIME
);

-- analyst action log (audit trail — also useful for a future feedback loop)
CREATE TABLE analyst_actions (
  action_id    INTEGER PRIMARY KEY AUTOINCREMENT,
  alert_id     TEXT REFERENCES alerts(alert_id),
  analyst_name TEXT,
  action       TEXT,      -- acknowledge | escalate | dismiss
  note        TEXT,
  action_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

5. Backend API (FastAPI)

Method	Endpoint	Purpose
POST	/flows/ingest	Accept a flow (from replay script or live capture), extract features
POST	/alerts/score	Run model + SHAP on a flow, create an alert if attack-flagged
GET	/alerts	List alerts, filterable by status/date/attack_type, sorted by confidence
GET	/alerts/{id}	Alert detail — SHAP breakdown + incident brief
POST	/alerts/{id}/action	Record acknowledge/escalate/dismiss
GET	/alerts/{id}/brief	Generate (or fetch cached) Azure OpenAI incident brief
GET	/model/info	Currently loaded model version, training date, eval metrics
GET	/health	Liveness check

Design notes: - `/alerts/score` is where detect → explain → prioritize happens in one call: model prediction, SHAP top-features, and a simple priority score (e.g. weighted by confidence + attack-type severity) computed before insert. - Keep the incident-brief call (`/alerts/{id}/brief`) separate and lazy (generate on first view, then cache in the `incident_brief` column) rather than on every single alert at ingest time — this keeps your Azure OpenAI usage and demo latency predictable, and is an honest “generated on demand” story rather than implying every alert is instantly LLM-annotated.

6. Azure OpenAI — Incident Brief

Prompt template (system + per-alert):

```
System: You are a SOC assistant. Given structured alert data, write a
3-4 sentence plain-English incident brief for a Tier-1 analyst. State
what was observed, why the model flagged it (using the given top
features), and a suggested next step. Do not invent details not
present in the input. Do not claim certainty the confidence score
does not support.

User: {
  "attack_type": "...",
  "confidence": 0.87,
  "top_features": [{"feature": "Flow Duration", "shap_value": 0.21}, ...],
  "src_ip": "...", "dst_ip": "...", "dst_port": 445
}
```

- Call via the Azure OpenAI REST/SDK endpoint (`azure-ai-openai` / `openai` Python SDK pointed at your Azure resource) — deployment name and endpoint come from environment variables, never hardcoded.
 - Guardrail: if the model’s own confidence is low, instruct the prompt to hedge language accordingly (“possible,” “worth reviewing”) rather than asserting certainty — this is a nice, cheap way to demonstrate you thought about LLM overclaiming, which is exactly the kind of thing a judge probing “what happens when the model is wrong” will appreciate.
-

7. Frontend (Dashboard)

Fast path (recommended given hackathon timeline): Streamlit - Page 1 — **Alert Queue**: table sorted by confidence, filters (status, attack type, time range), color-coded severity. - Page 2 — **Alert Detail**: SHAP bar chart (top 5 features), raw flow metadata, incident brief text, Acknowledge/Escalate/Dismiss buttons. - Page 3 — **Model Info**: current model version, eval metrics, dataset/date trained — this single page does a lot of work convincing a judge the system is real.

Polish path (if time allows): React - Same three views, calling the same FastAPI endpoints — build Streamlit first, port to React only if the team has spare time before the demo, not as parallel effort.

8. Live/Replay Demo Pipeline

1. Select or synthesize a PCAP containing one attack not well-represented in training (e.g. a modified/parameter-shifted variant of an existing attack type) — this is your “signature-evading” moment. **Be precise in your own language about what this demonstrates**: a variant not well-represented in the training distribution, not a validated real-world zero-day.
2. Run CICFlowMeter on the PCAP → flow CSV.
3. Replay script reads the CSV row-by-row (with artificial delay to simulate real time) and POSTs to `/flows/ingest`.
4. Dashboard, open in a second window, shows the alert appear live in the queue, confidence + SHAP explanation populate, incident brief generates on click, analyst clicks Escalate.
5. Have a **pre-recorded screen capture as a fallback** in case live Azure OpenAI calls or Wi-Fi fail during the actual judging — this is standard hackathon risk management, not overselling.

9. Full Task Breakdown

Phase 0 — Project Setup

- ☐ Repo structure: `/data`, `/ml_pipeline`, `/api`, `/dashboard`, `/notebooks`, `/scripts`, `/docs`
- ☐ `requirements.txt` split into `requirements-ml.txt` (training-only: scikit-learn, imbalanced-learn, shap) and `requirements-api.txt` (serving-only: fastapi, uvicorn, joblib, shap, openai)
- ☐ `.env.example` for Azure ML workspace name/subscription, Azure OpenAI endpoint/key/deployment
- ☐ Azure ML workspace provisioned
- ☐ Azure OpenAI resource provisioned + deployment created

Phase 1 — Data & Preprocessing

- ☐ Download CICIDS2017, verify file integrity/row counts against published dataset stats
- ☐ EDA notebook: class balance, missing values, feature distributions
- ☐ Cleaning script: drop leakage columns, handle NaN/Infinity, dedupe
- ☐ Day-based train/val/test split implemented and saved as fixed indices (so every later run uses the identical split)
- ☐ Feature scaler fit on train split only, saved

Phase 2 — Model Training

- ☐ Baseline RF (binary) trained, metrics logged
- ☐ SMOTE applied to train split only, re-trained, compare metrics before/after
- ☐ Hyperparameter search (time-aware CV) for binary model
- ☐ Multiclass RF trained on attack subtype labels (rare classes merged/handled)
- ☐ Per-class precision/recall/F1/AUC report generated and saved (not just printed in a notebook)
- ☐ Confusion matrices saved as image artifacts

Phase 3 — Explainability

- ☐ SHAP TreeExplainer fit on final binary model
- ☐ SHAP TreeExplainer fit on final multiclass model
- ☐ Global feature importance chart generated
- ☐ Per-alert local explanation function: input feature vector → top-5 `{feature, shap_value}`
- ☐ Explainer serialized alongside model artifacts

Phase 4 — Azure ML Registration

- ☐ Model artifacts (binary + multiclass + scaler + explainers) packaged
- ☐ Registered in Azure ML with metadata tags (dataset version, split date, eval metrics)
- ☐ Confirm model can be pulled by name/version from a clean environment (not just the training machine)

Phase 5 — Backend API

- ☐ FastAPI skeleton + `/health` endpoint
- ☐ SQLite schema created, migration script
- ☐ Feature extraction function shared between training and serving (same code path — avoids train/serve skew)
- ☐ `/flows/ingest` implemented
- ☐ `/alerts/score` implemented: model load → predict → SHAP explain → priority score → DB insert
- ☐ `/alerts`, `/alerts/{id}` GET endpoints
- ☐ `/alerts/{id}/action` implemented, writes to `analyst_actions`
- ☐ `/model/info` implemented

Phase 6 — Azure OpenAI Integration

- ☐ Prompt template finalized and tested against 5–10 varied alert examples for tone/accuracy
- ☐ `/alerts/{id}/brief` endpoint: generate-on-demand, cache in DB
- ☐ Low-confidence hedging behavior verified
- ☐ Error handling: brief generation failure should not block the alert from appearing in the queue

Phase 7 — Dashboard

- ☐ Streamlit app skeleton, connects to FastAPI
- ☐ Alert Queue page: table, sort, filter, severity color-coding
- ☐ Alert Detail page: SHAP chart, flow metadata, brief text, action buttons
- ☐ Model Info page
- ☐ Basic auth/analyst-name entry (even a simple text field) so `analyst_actions` has real attribution

Phase 8 — Replay/Demo Pipeline

- ☐ Candidate demo PCAP selected/prepared
- ☐ CICFlowMeter run, output verified
- ☐ Replay script built (CSV → paced POSTs to `/flows/ingest` → `/alerts/score`)
- ☐ Full dry run: replay → dashboard → brief → escalate, timed end-to-end
- ☐ Screen-recorded fallback captured

Phase 9 — Testing & Hardening

- ☐ Unit tests: feature extraction consistency (same input → same output across train/serve paths)
- ☐ API tests: each endpoint, happy path + error cases (malformed flow, missing model)
- ☐ Load check: dashboard usable with a few hundred alerts in the queue (pagination if needed)

Phase 10 — Documentation & Submission Prep

- ☐ README: setup steps, how to run training, how to run the API, how to run the demo
- ☐ Model card: dataset, split strategy, metrics, known limitations
- ☐ Architecture diagram (this document) kept in `/docs`
- ☐ Final Round 1 deck cross-checked against actual implementation status (no claim in the deck should outpace what’s actually built)

10. Suggested Team Split (adjust to your actual roster)

Track	Owns
ML Engineer	Phases 1–4 (data, model, SHAP, Azure ML registry)
Backend/Cloud	Phases 5–6 (FastAPI, DB, Azure OpenAI integration)
Frontend	Phase 7 (dashboard)
Cybersecurity/Research	Phase 8 (demo scenario selection, attack variant justification) + honest metric review across Phases 2 and 9

All four tracks converge in Phase 8 — the replay demo is the integration point, so schedule a full dry run at least 2–3 days before Round 2, not the night before.